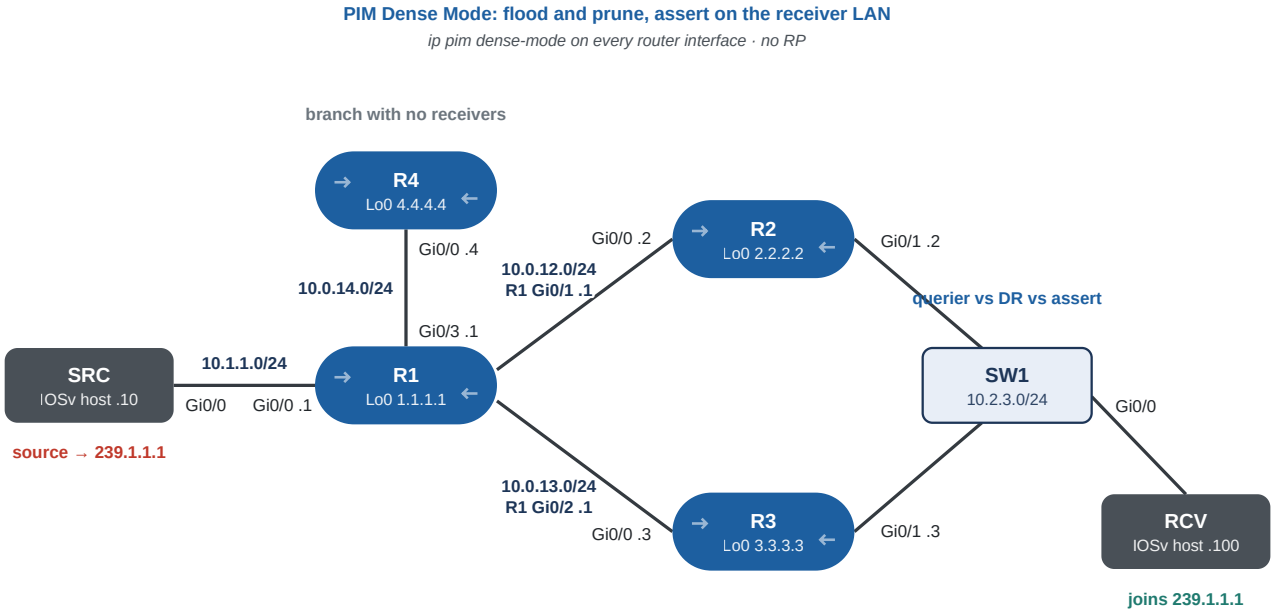
PIM Lab 01: Multicast - PIM Dense Mode

Lab Objective:

The focus of this lab is to understand IP Multicast PIM Dense Mode forwarding in Cisco IOS routers. Additional technologies include IGMP, the IGMP querier and PIM DR elections, flood and prune, the PIM assert mechanism and PIM Dense Mode State Refresh.

Lab Topology:

The lab network topology is illustrated below:



IMPORTANT NOTE

Build the topology in CML with six **IOSv** nodes and one **unmanaged switch**. SRC and RCV are IOSv routers acting as plain hosts (`no ip routing`): SRC sends the multicast stream, and RCV joins groups with `ip igmp join-group` and answers pings sent to them. All five labs in this book use this topology and build on the previous lab.

```
R1:  Gi0/0 10.1.1.1/24   Gi0/1 10.0.12.1/24   Gi0/2 10.0.13.1/24   Gi0/3 10.0.14.1/24   Lo0
1.1.1.1/32
R2:  Gi0/0 10.0.12.2/24  Gi0/1 10.2.3.2/24                                              Lo0 2.2.2.2/32
R3:  Gi0/0 10.0.13.3/24  Gi0/1 10.2.3.3/24                                              Lo0 3.3.3.3/32
R4:  Gi0/0 10.0.14.4/24                                                                Lo0 4.4.4.4/32
SRC: Gi0/0 10.1.1.10/24    default gateway 10.1.1.1
RCV: Gi0/0 10.2.3.100/24   default gateway 10.2.3.2
```

Base configuration for the hosts and the IGP (set the OSPF router ID by hand, Lab 05 needs it):

```
SRC(config)#no ip routing
SRC(config)#ip default-gateway 10.1.1.1
R1(config)#router ospf 1
R1(config-router)#router-id 1.1.1.1
R1(config-router)#network 0.0.0.0 255.255.255.255 area 0
```

```
R2(config-router)#passive-interface GigabitEthernet0/1
R3(config-router)#passive-interface GigabitEthernet0/1
```

The receiver LAN is passive on R2 and R3 (no OSPF adjacency across it), so it never becomes a transit path and every RPF check upstream points toward R1.

When this book says "SRC sends to G", run a long ping on SRC: `ping 239.1.1.1 repeat 100000`, and leave it running in its own console tab. Stop it with Ctrl+Shift+6.

Task 1

Configure the hostnames, IP addressing and OSPF area 0 on R1, R2, R3 and R4, and configure SRC and RCV as hosts as shown above. Verify that SRC can ping RCV.

Task 2

Enable multicast routing on R1, R2, R3 and R4, and enable PIM Dense Mode on all of their interfaces, including Loopback0. Verify the PIM neighbors. Which router is the PIM Designated Router (DR) on the 10.2.3.0/24 segment, and why?

Task 3

Configure RCV to join group 239.1.1.1. Which router is the IGMP querier on the 10.2.3.0/24 segment? Explain why the querier and the DR are not the same router.

Task 4

SRC sends to 239.1.1.1. Verify that RCV answers. Examine the multicast routing table of R1 and R4, and explain the flood and prune behavior. Which routers hold (S,G) state for this stream?

Task 5

Both R2 and R3 receive the stream from R1, and both are connected to the receiver LAN. Verify which router forwards the stream onto 10.2.3.0/24 and which one doesn't. Explain how that was decided.

Task 6

Without changing any PIM configuration, make R2 the router that forwards the stream onto the receiver LAN. Verify your solution.

Task 7

Watch the pruned interface toward R4 on R1 for a few minutes and explain what happens when its timer expires. Then configure R1 so that the network stops this periodic re-flooding. Verify your solution.

Lab Validation:

The outputs below are trimmed. Timers, counters and uptimes will differ in your lab.

Task 1

```
R1(config)#hostname R1
R1(config)#interface GigabitEthernet0/0
R1(config-if)#ip address 10.1.1.1 255.255.255.0
R1(config-if)#no shutdown
```

Repeat for every interface in the table, then OSPF as shown in the note. The hosts:

```
SRC(config)#no ip routing
SRC(config)#ip default-gateway 10.1.1.1
SRC(config)#interface GigabitEthernet0/0
SRC(config-if)#ip address 10.1.1.10 255.255.255.0
SRC(config-if)#no shutdown
SRC#ping 10.2.3.100
!!!!!
```

Task 2

```
R1(config)#ip multicast-routing
R1(config)#interface range GigabitEthernet0/0 - 3 , Loopback0
R1(config-if-range)#ip pim dense-mode
```

The same on R2, R3 and R4 with their own interfaces.

```
R2#show ip pim neighbor
Neighbor          Interface               Uptime/Expires    Ver   DR
Address                                                           Prio/Mode
10.0.12.1         GigabitEthernet0/0      00:02:14/00:01:28 v2    1 / S P G
10.2.3.3          GigabitEthernet0/1      00:02:01/00:01:41 v2    1 / DR S P G
```

R3 (10.2.3.3) is the DR on the receiver LAN: both routers have the default DR priority 1, so the **highest IP address** wins. Hellos are sent every 30 seconds to 224.0.0.13. The mode letters **S P G** mean State Refresh capable, DR Priority capable and GenID capable. In Dense Mode the DR has little to do (it matters for IGMPv1 and for Sparse Mode).

Task 3

```
RCV(config)#interface GigabitEthernet0/0
RCV(config-if)#ip igmp join-group 239.1.1.1
```

```
R2#show ip igmp interface GigabitEthernet0/1 | include IGMP|router
  IGMP is enabled on interface
  Current IGMP host version is 2
  Current IGMP router version is 2
  IGMP query interval is 60 seconds
  IGMP querier timeout is 120 seconds
  IGMP activity: 1 joins, 0 leaves
  Multicast routing is enabled on interface
  Multicast designated router (DR) is 10.2.3.3
  IGMP querying router is 10.2.3.2 (this system)
```

- **IGMP querier = R2 (10.2.3.2)**: the **lowest** IP address wins the IGMPv2 querier election.
- **PIM DR = R3 (10.2.3.3)**: the **highest** IP (or highest DR priority) wins the PIM DR election.

They are two different protocols with opposite tie-breakers, so on a LAN with two routers they usually end up on different routers. The querier sends the General Queries (every 60 seconds on IOS); the non-querier takes over if it hears none for the querier timeout (2 × query interval = 120 seconds).

Task 4

```
SRC#ping 239.1.1.1 repeat 100000
Type escape sequence to abort.
Sending 100000, 100-byte ICMP Echos to 239.1.1.1, timeout is 2 seconds:

Reply to request 0 from 10.2.3.100, 6 ms
Reply to request 0 from 10.2.3.100, 7 ms
Reply to request 1 from 10.2.3.100, 3 ms
```

The duplicate reply to request 0 is the flooding at work: both R2 and R3 forwarded the first packet onto the LAN (Task 5 explains how that stops).

```
R1#show ip mroute 239.1.1.1
(*, 239.1.1.1), 00:01:45/stopped, RP 0.0.0.0, flags: D
  Incoming interface: Null, RPF nbr 0.0.0.0
  Outgoing interface list:
    GigabitEthernet0/3, Forward/Dense, 00:01:45/stopped
    GigabitEthernet0/2, Forward/Dense, 00:01:45/stopped
    GigabitEthernet0/1, Forward/Dense, 00:01:45/stopped

(10.1.1.10, 239.1.1.1), 00:01:45/00:02:58, flags: T
  Incoming interface: GigabitEthernet0/0, RPF nbr 0.0.0.0
  Outgoing interface list:
    GigabitEthernet0/1, Prune/Dense, 00:01:40/00:01:19
    GigabitEthernet0/3, Prune/Dense, 00:01:40/00:01:19
    GigabitEthernet0/2, Forward/Dense, 00:01:45/stopped

R4#show ip mroute 239.1.1.1 | begin \(10
(10.1.1.10, 239.1.1.1), 00:01:44/00:01:15, flags: PT
  Incoming interface: GigabitEthernet0/0, RPF nbr 10.0.14.1
  Outgoing interface list: Null
```

- **Flood**: the first packet was forwarded out **every** PIM interface of R1. Dense Mode assumes everyone wants the traffic.
- **Prune**: R4 has no receivers and no downstream neighbors, so it sent a **PIM Prune** upstream. R1 shows Gi0/3 as **Prune/Dense**, and R4's (S,G) entry has flag **P** with an empty outgoing list. R2 pruned too, after losing the assert (Task 5).
- **State**: **every** router, R4 included, holds (S,G) state for 10.1.1.10, even when pruned. Dense Mode keeps state everywhere until the source stops.
- The (*,G) entry with RP 0.0.0.0 is only a parent entry. Dense Mode forwards on the **(S,G) source tree**.

Task 5

```
R3#show ip mroute 239.1.1.1 | begin \(10
(10.1.1.10, 239.1.1.1), 00:03:02/00:02:58, flags: T
  Incoming interface: GigabitEthernet0/0, RPF nbr 10.0.13.1
  Outgoing interface list:
    GigabitEthernet0/1, Forward/Dense, 00:03:02/stopped, A

R2#show ip mroute 239.1.1.1 | begin \(10
(10.1.1.10, 239.1.1.1), 00:03:01/00:01:59, flags: PT
  Incoming interface: GigabitEthernet0/0, RPF nbr 10.0.12.1
```

```
  Outgoing interface list:
    GigabitEthernet0/1, Prune/Dense, 00:02:58/00:00:01
```

R3 forwards and R2 doesn't. Both routers forwarded the first packets onto the LAN. Each saw the other's copy arrive on an interface in its own outgoing list, so both sent a **PIM Assert** with their route to the source:

	Administrative distance	Metric to 10.1.1.0/24	IP address
R2	110 (OSPF)	2	10.2.3.2
R3	110 (OSPF)	2	**10.2.3.3**

Lowest AD wins, then lowest metric, then **highest IP**. Here it's a full tie until the last step, so **R3 wins** (the **A** after its interface = assert winner) and R2 prunes its LAN interface. With nothing left to forward to, R2 also pruned upstream toward R1.

Task 6

The assert compares routes to the source, so make R2's route better by raising R3's OSPF cost toward R1:

```
R3(config)#interface GigabitEthernet0/0
R3(config-if)#ip ospf cost 10
```

R3's metric to 10.1.1.0/24 is now 11, while R2 stays at 2. A new assert happens only when duplicates appear again, so clear the state on both routers to trigger it at once:

```
R2#clear ip mroute *
R3#clear ip mroute *
R2#show ip mroute 239.1.1.1 | include GigabitEthernet0/1
    GigabitEthernet0/1, Forward/Dense, 00:00:21/stopped, A
R3#show ip mroute 239.1.1.1 | include GigabitEthernet0/1
    GigabitEthernet0/1, Prune/Dense, 00:00:19/00:02:40
```

R2 now wins on metric (2 < 11), so the IP address no longer matters. This only works because the LAN is passive in OSPF: otherwise R3 would simply route to the source across the LAN through R2. Remove the cost again before the next lab, so that R2 and R3 are equal again:

```
R3(config-if)#no ip ospf cost 10
```

Task 7

Watch the expiry timer of the pruned interface:

```
R1#show ip mroute 239.1.1.1 | include 0/3
    GigabitEthernet0/3, Prune/Dense, 00:02:52/00:00:07
R1#show ip mroute 239.1.1.1 | include 0/3
    GigabitEthernet0/3, Forward/Dense, 00:00:02/stopped
R4#show ip mroute count | begin 239.1.1.1
Group: 239.1.1.1, Source count: 1, Packets forwarded: 0, Packets received: 7
```

A prune lasts **3 minutes**. When it expires, R1 puts Gi0/3 back to Forward and **floods R4 again**, R4 prunes again, and the cycle repeats every 3 minutes for as long as the source sends: the received counter on R4 grows in small jumps.

State Refresh (RFC 3973) stops this. The first-hop router periodically sends State Refresh messages down the tree, and every router that receives one resets its prune timers instead of letting them expire. All IOS routers process and forward State Refresh by default, but only the first-hop router can originate it, and that is off by default. Enable it on R1's source-facing interface:

```
R1(config)#interface GigabitEthernet0/0
R1(config-if)#ip pim state-refresh origination-interval 60
```

```
R1#show ip mroute 239.1.1.1 | include 0/3
    GigabitEthernet0/3, Prune/Dense, 00:08:41/00:02:58
```

The prune keeps being refreshed (the expiry timer jumps back up every 60 seconds), and R4's received counter stops growing. Dense Mode still floods once at the start and still keeps (S,G) state on every router, which is why it isn't used in production.

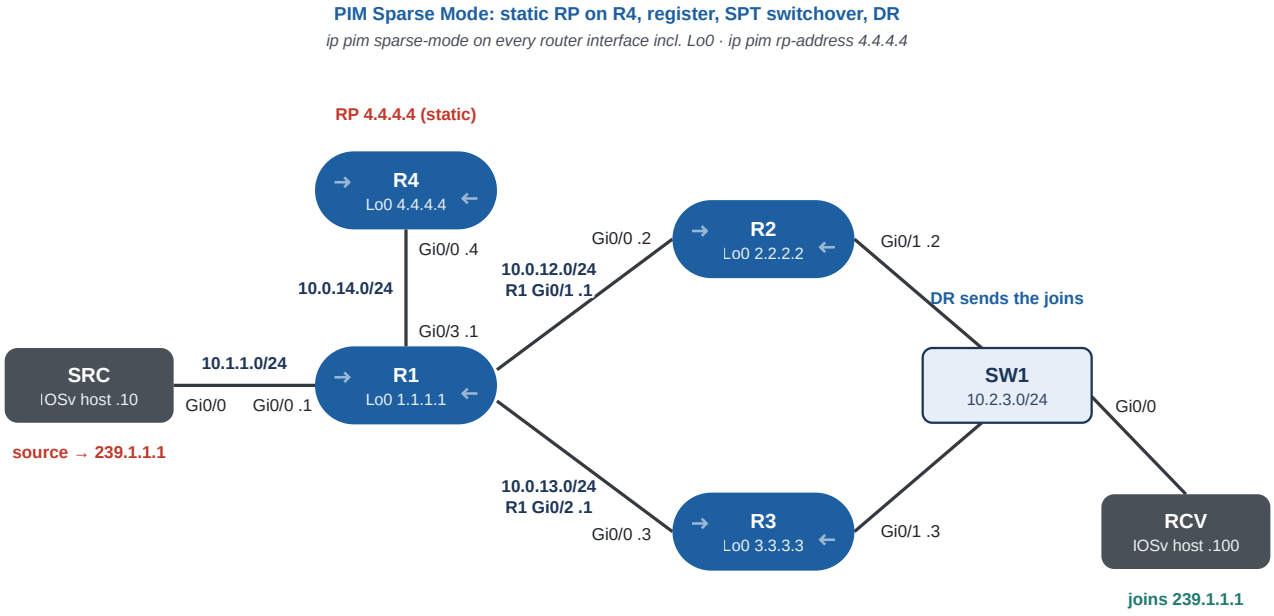
PIM Lab 02: Multicast - PIM Sparse Mode

Lab Objective:

The focus of this lab is to understand IP Multicast PIM Sparse Mode forwarding in Cisco IOS routers. Additional technologies include static Rendezvous Point configuration, the shared tree, source registration with Register and Register-Stop messages, SPT switchover and the PIM Designated Router.

Lab Topology:

The lab network topology is illustrated below:



IMPORTANT NOTE

Continue from the end of PIM Lab 01. Stop the ping on SRC before you start. RCV is still joined to 239.1.1.1, and R2 and R3 again have equal OSPF costs toward R1.

Task 1

Change all PIM interfaces of R1, R2, R3 and R4 (including Loopback0) to Sparse Mode, and remove the State Refresh configuration from R1. Configure R4's Loopback0 as the static RP for all groups on every router. Verify the group-to-RP mapping and the PIM register tunnels that the routers created.

Task 2

Without any source sending, verify the multicast routing state for 239.1.1.1 on R3, R2, R1 and R4. Draw the shared tree on paper. Which router on the receiver LAN sent the PIM Join, and why that one?

Task 3

SRC sends to 239.1.1.1. Verify that RCV answers. Verify the source registration on R1 and R4, and the SPT switchover on the last-hop router. Explain the meaning of the flags you see on each router.

Task 4

Configure the last-hop router so that it never switches over to the shortest-path tree. Verify the result, and explain why the traffic path looks the same in this topology.

Task 5

Make R2 the PIM DR on the receiver LAN without changing any IP address. Verify which router now sends the joins toward the RP.

Task 6

Stop the receiver: RCV leaves 239.1.1.1 while SRC keeps sending. Verify what R4 and R1 do with the stream now, and how R1 checks whether the RP has new receivers.

Lab Validation:

The outputs below are trimmed. Timers, counters and uptimes will differ in your lab.

Task 1

```
R1(config)#interface range GigabitEthernet0/0 - 3 , Loopback0
R1(config-if-range)#ip pim sparse-mode
R1(config-if-range)#interface GigabitEthernet0/0
R1(config-if)#no ip pim state-refresh origination-interval
R1(config-if)#exit
R1(config)#ip pim rp-address 4.4.4.4
```

`ip pim sparse-mode` replaces `ip pim dense-mode` on each interface. Do the same on R2, R3 and R4, and configure the same `ip pim rp-address 4.4.4.4` on **every** router, R4 included. PIM must run on R4's Loopback0, because that is the RP address.

```
R3#show ip pim rp mapping
PIM Group-to-RP Mappings

Group(s): 224.0.0.0/4, Static
    RP: 4.4.4.4 (?)

R1#show ip pim tunnel
Tunnel0
  Type       : PIM Encap
  RP         : 4.4.4.4
  Source     : 10.0.14.1
R4#show ip pim tunnel
Tunnel0
  Type       : PIM Encap
  RP         : 4.4.4.4
  Source     : 4.4.4.4
Tunnel1*
  Type       : PIM Decap
```

```
  RP         : 4.4.4.4
  Source     : -
```

Every router with an RP address builds an **encapsulation tunnel** toward the RP, used to send Register messages. The RP also builds a **decapsulation tunnel** to receive them. These tunnels stay up as long as the RP is reachable, even with no streams.

Task 2

```
R3#show ip mroute 239.1.1.1
(*, 239.1.1.1), 00:01:31/00:02:28, RP 4.4.4.4, flags: SJC
  Incoming interface: GigabitEthernet0/0, RPF nbr 10.0.13.1
  Outgoing interface list:
    GigabitEthernet0/1, Forward/Sparse, 00:01:31/00:02:28

R1#show ip mroute 239.1.1.1
(*, 239.1.1.1), 00:01:30/00:03:00, RP 4.4.4.4, flags: S
  Incoming interface: GigabitEthernet0/3, RPF nbr 10.0.14.4
  Outgoing interface list:
    GigabitEthernet0/2, Forward/Sparse, 00:01:30/00:03:00

R4#show ip mroute 239.1.1.1
(*, 239.1.1.1), 00:01:29/00:03:01, RP 4.4.4.4, flags: S
  Incoming interface: Null, RPF nbr 0.0.0.0
  Outgoing interface list:
    GigabitEthernet0/0, Forward/Sparse, 00:01:29/00:03:01
```

Shared tree: **R4 (RP) → R1 → R3 → LAN**. The state was created by an **explicit join**, without any traffic. That is the opposite of Dense Mode.

R3 sent the join because it is the PIM DR on the LAN (highest IP, 10.2.3.3). R2 is the IGMP querier and sees the same report, but only the DR joins, so the LAN doesn't receive the stream twice. The join went out of R3's **RPF interface toward the RP** (Gi0/0, its unicast route to 4.4.4.4). The incoming interface on R4 is Null: the RP is the root of the shared tree.

Task 3

```
SRC#ping 239.1.1.1 repeat 100000
Reply to request 0 from 10.2.3.100, 12 ms
Reply to request 1 from 10.2.3.100, 3 ms
```

```
R1#show ip mroute 239.1.1.1 | begin \(10
(10.1.1.10, 239.1.1.1), 00:00:41/00:02:58, flags: FT
  Incoming interface: GigabitEthernet0/0, RPF nbr 0.0.0.0
  Outgoing interface list:
    GigabitEthernet0/2, Forward/Sparse, 00:00:39/00:02:52

R4#show ip mroute 239.1.1.1 | begin \(10
(10.1.1.10, 239.1.1.1), 00:00:41/00:02:18, flags: P
  Incoming interface: GigabitEthernet0/0, RPF nbr 10.0.14.1
  Outgoing interface list: Null

R3#show ip mroute 239.1.1.1
(*, 239.1.1.1), 00:09:12/stopped, RP 4.4.4.4, flags: SJC
  Incoming interface: GigabitEthernet0/0, RPF nbr 10.0.13.1
  Outgoing interface list:
    GigabitEthernet0/1, Forward/Sparse, 00:09:12/00:02:41

(10.1.1.10, 239.1.1.1), 00:00:40/00:02:19, flags: JT
  Incoming interface: GigabitEthernet0/0, RPF nbr 10.0.13.1
```

```
  Outgoing interface list:
    GigabitEthernet0/1, Forward/Sparse, 00:00:40/00:02:41
```

What happened, in order:

1. R1, the **first-hop router**, received the first packet. It encapsulated it in a unicast **Register** message to 4.4.4.4 (flag **F** = this router registers the source).
2. R4, the **RP**, decapsulated it, sent it down the shared tree, and sent an **(S,G) Join** toward the source to receive the stream natively.
3. R3, the **last-hop router**, received the first packet over the shared tree, learned the source address, and immediately sent an **(S,G) Join toward 10.1.1.10**: that is the **SPT switchover** (flag **J** = joined the SPT, **T** = traffic now arriving on the SPT).
4. R1 now forwards the (S,G) stream directly to R3. R4 doesn't need it anymore, so its (S,G) is **pruned** (flag **P**), and R4 sent a **Register-Stop** to R1.

Final path: **SRC → R1 → R3 → RCV**, without passing through the RP.

Task 4

```
R3(config)#ip pim spt-threshold infinity
R3#clear ip mroute *
R3#show ip mroute 239.1.1.1
(*, 239.1.1.1), 00:00:38/00:02:21, RP 4.4.4.4, flags: SC
  Incoming interface: GigabitEthernet0/0, RPF nbr 10.0.13.1
  Outgoing interface list:
    GigabitEthernet0/1, Forward/Sparse, 00:00:38/00:02:21
```

R3 now keeps **only the (*,G) entry**: no (S,G) and no **J** flag, so it stays on the shared tree. You can also limit this to some groups with `ip pim spt-threshold infinity group-list <acl>`.

The path still looks like SRC → R1 → R3, because **R1 sits on both trees**. The shared tree from R4 to R3 passes through R1, and R1 already has the source on its directly connected (S,G) entry, so it copies the traffic straight onto the shared-tree branch toward R3. The RP is on a side branch here. In a topology where the RP is in the middle of the path, the difference would be visible. Remove the command before continuing:

```
R3(config)#no ip pim spt-threshold infinity
```

Task 5

```
R2(config)#interface GigabitEthernet0/1
R2(config-if)#ip pim dr-priority 100
```

```
R3#show ip pim neighbor GigabitEthernet0/1
Neighbor          Interface               Uptime/Expires    Ver   DR
Address                                                           Prio/Mode
10.2.3.2          GigabitEthernet0/1      01:02:11/00:01:33 v2    100/ DR S P G

R2#show ip mroute 239.1.1.1 | include flags|Incoming|0/1
(*, 239.1.1.1), 00:00:52/00:02:07, RP 4.4.4.4, flags: SJC
  Incoming interface: GigabitEthernet0/0, RPF nbr 10.0.12.1
    GigabitEthernet0/1, Forward/Sparse, 00:00:52/00:02:07
```

The highest **DR priority** wins before the IP address is compared, so R2 became DR and now sends the (*,G) and (S,G) joins for the LAN. R3 stops forwarding the group onto the LAN.

Task 6

```
RCV(config)#interface GigabitEthernet0/0
RCV(config-if)#no ip igmp join-group 239.1.1.1
```

RCV sends an IGMPv2 **Leave** to 224.0.0.2. The querier (R2) answers with a group-specific query, nobody replies, and the outgoing list empties. The routers prune back toward the source.

```
R4#show ip mroute 239.1.1.1
(*, 239.1.1.1), 01:10:12/00:02:51, RP 4.4.4.4, flags: SP
  Incoming interface: Null, RPF nbr 0.0.0.0
  Outgoing interface list: Null

R1#show ip mroute 239.1.1.1 | begin \(10
(10.1.1.10, 239.1.1.1), 00:42:15/00:02:52, flags: PFT
  Incoming interface: GigabitEthernet0/0, RPF nbr 0.0.0.0
  Outgoing interface list: Null
```

R1 drops the stream (outgoing list Null, flag **P**). It still owns the source (flag **F**), so it periodically sends a **Null-Register** (a Register without data) to the RP to ask whether there are new receivers. R4 answers each one with a **Register-Stop** while its outgoing list stays empty. Watch the exchange with `debug ip pim`. When RCV joins again, R4 sends an (S,G) join toward R1 and the stream flows again. Rejoin RCV before the next lab:

```
RCV(config-if)#ip igmp join-group 239.1.1.1
```

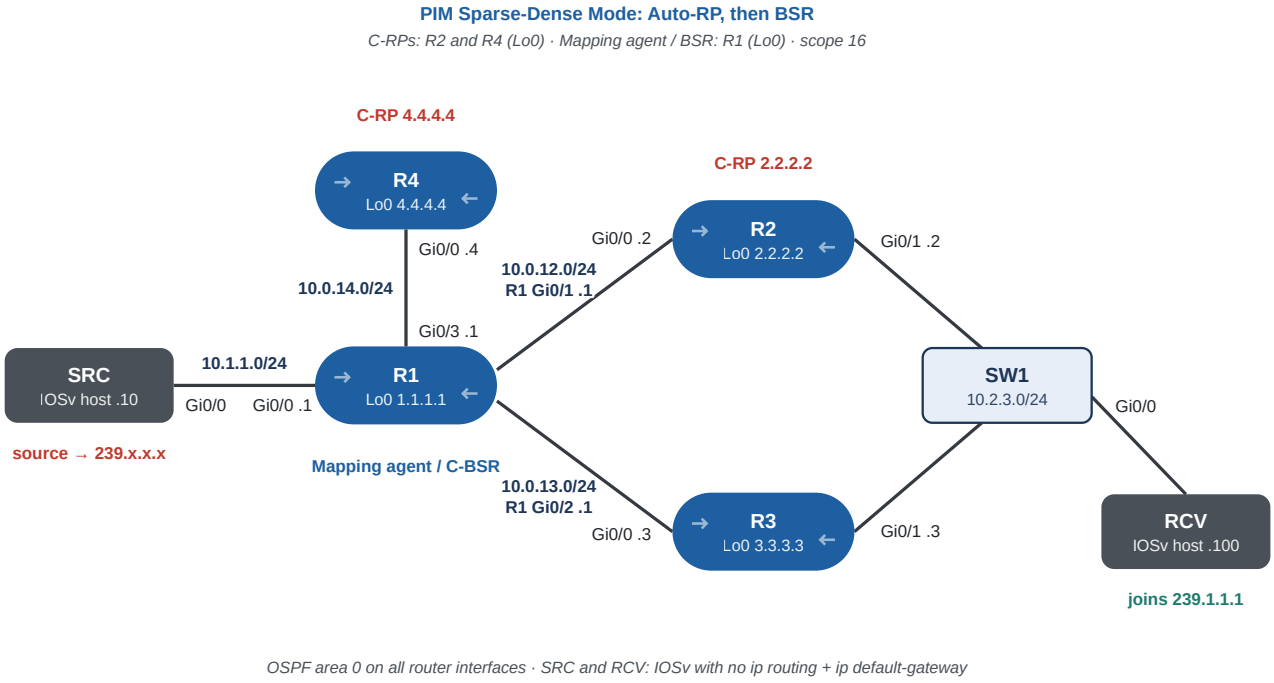
PIM Lab 03: Multicast - PIM Sparse-Dense Mode, Auto-RP and BSR

Lab Objective:

The focus of this lab is to understand PIM Sparse-Dense Mode and dynamic RP discovery in Cisco IOS routers. Additional technologies include Cisco Auto-RP candidate RPs and mapping agents, RP selection per group range, dense-mode fallback, the Auto-RP listener, and the PIM Bootstrap Router (BSR).

Lab Topology:

The lab network topology is illustrated below:



OSPF area 0 on all router interfaces · SRC and RCV: IOSv with no ip routing + ip default-gateway

IMPORTANT NOTE

Continue from the end of PIM Lab 02: RCV is joined to 239.1.1.1 and R2 is the DR on the receiver LAN. Auto-RP and BSR were not designed to work together. This lab uses them one after the other, never at the same time.

Task 1

Remove the static RP from all routers. Change all PIM interfaces of R1, R2, R3 and R4, including Loopback0, to Sparse-Dense Mode.

Task 2

Configure Cisco Auto-RP as follows:

- R2 and R4 announce their Loopback0 as candidate RP for all groups
- R1 is the mapping agent and uses its Loopback0

- Announcements and discoveries must not travel more than 16 hops

Verify which RP the routers use and how they learned it. Verify the multicast routing entries that Auto-RP itself uses, and explain why they are in Dense Mode.

Task 3

R2 should only be candidate RP for groups 239.2.0.0/16. R4 stays candidate RP for all groups. Verify the group-to-RP mapping on R3. Configure RCV to also join 239.2.2.2, send traffic to it from SRC, and verify which RP the receiver-side DR uses for each of the two groups.

Task 4

R4 stops being a candidate RP. Wait for the mapping to expire and verify the RP mapping again. With SRC sending to 239.1.1.1, verify how the routers now forward that group, and explain why.

Task 5

Convert the network to the recommended Auto-RP design: all PIM interfaces back to Sparse Mode, and only the two Auto-RP groups allowed to run in Dense Mode. Restore R4 as candidate RP for all groups. Verify that Auto-RP still works and that 239.1.1.1 no longer falls back to Dense Mode.

Task 6

Replace Auto-RP with the PIM Bootstrap Router mechanism:

- R1 is the candidate BSR, using Loopback0
- R2 and R4 are candidate RPs for all groups, using Loopback0
- R4 must be the preferred RP

Verify the elected BSR, the RP set, and the RP the routers use for 239.1.1.1.

Lab Validation:

The outputs below are trimmed. Timers, counters and uptimes will differ in your lab.

Task 1

```
R1(config)#no ip pim rp-address 4.4.4.4
R1(config)#interface range GigabitEthernet0/0 - 3 , Loopback0
R1(config-if-range)#ip pim sparse-dense-mode
```

The same on R2, R3 and R4. With no RP known yet, every group falls back to Dense Mode: **sparse-dense-mode** means Sparse for groups with an RP, Dense for groups without one.

Task 2

```
R2(config)#ip pim send-rp-announce Loopback0 scope 16
R4(config)#ip pim send-rp-announce Loopback0 scope 16
R1(config)#ip pim send-rp-discovery Loopback0 scope 16
```

Command	Meaning
`send-rp-announce Loopback0 scope 16`	Candidate RP: announce Lo0 as RP every 60 s to **224.0.1.39** (Cisco-RP-Announce), TTL 16
`send-rp-discovery Loopback0 scope 16`	Mapping agent: listen on 224.0.1.39, pick the RP per group range, advertise the result to **224.0.1.40** (Cisco-RP-Discovery)
`scope 16`	The **TTL** of the Auto-RP packets, so the hop limit

```
R1#show ip pim rp mapping
PIM Group-to-RP Mappings
This system is an RP-mapping agent (Loopback0)

Group(s) 224.0.0.0/4
  RP 4.4.4.4 (?), v2v1
    Info source: 4.4.4.4 (?), elected via Auto-RP
         Uptime: 00:01:52, expires: 00:02:07
  RP 2.2.2.2 (?), v2v1
    Info source: 2.2.2.2 (?), via Auto-RP
         Uptime: 00:01:55, expires: 00:02:04

R3#show ip pim rp mapping
Group(s) 224.0.0.0/4
  RP 4.4.4.4 (?), v2v1
    Info source: 1.1.1.1 (?), elected via Auto-RP
         Uptime: 00:01:40, expires: 00:02:19
```

- The mapping agent heard **both** candidates, and **4.4.4.4 was elected**: for the same group range, the **highest IP** wins.
- R3 only received the result, from the mapping agent (**Info source 1.1.1.1**).
- The mapping expires after 3 × the announce interval (about 3 minutes) unless it's refreshed.

```
R3#show ip mroute 224.0.1.40
(*, 224.0.1.40), 00:05:11/stopped, RP 0.0.0.0, flags: DCL
  Incoming interface: Null, RPF nbr 0.0.0.0
  Outgoing interface list:
    GigabitEthernet0/1, Forward/Sparse-Dense, 00:05:11/stopped
    GigabitEthernet0/0, Forward/Sparse-Dense, 00:05:11/stopped
```

The Auto-RP groups **224.0.1.39 and 224.0.1.40** run in **Dense Mode** (flag **D**, RP 0.0.0.0). That's the chicken-and-egg problem Sparse-Dense Mode solves: routers need the RP information to run Sparse Mode, but they can only receive it if those two groups reach them **without** an RP. So they're flooded. Every PIM router joins 224.0.1.40 automatically (flag **L**).

Task 3

```
R2(config)#ip access-list standard AUTORP-R2
R2(config-std-nacl)#permit 239.2.0.0 0.0.255.255
R2(config-std-nacl)#exit
R2(config)#ip pim send-rp-announce Loopback0 scope 16 group-list AUTORP-R2
RCV(config)#interface GigabitEthernet0/0
RCV(config-if)#ip igmp join-group 239.2.2.2
SRC#ping 239.2.2.2 repeat 5
Reply to request 0 from 10.2.3.100, 8 ms
```

```
R3#show ip pim rp mapping
Group(s) 224.0.0.0/4
  RP 4.4.4.4 (?), v2v1
```

```
    Info source: 1.1.1.1 (?), elected via Auto-RP
Group(s) 239.2.0.0/16
  RP 2.2.2.2 (?), v2v1
    Info source: 1.1.1.1 (?), elected via Auto-RP

R2#show ip mroute 239.2.2.2 | include RP
(*, 239.2.2.2), 00:00:49/stopped, RP 2.2.2.2, flags: SJC
R2#show ip mroute 239.1.1.1 | include RP
(*, 239.1.1.1), 01:31:02/stopped, RP 4.4.4.4, flags: SJC
```

The more specific range wins: **239.2.0.0/16 → RP 2.2.2.2**, everything else → RP 4.4.4.4. That's the Auto-RP strength from the notes: **different RPs for different group ranges**, for load splitting. R2 is the DR on the LAN since Lab 02, so it is the router that joins.

Task 4

```
R4(config)#no ip pim send-rp-announce Loopback0 scope 16
```

After about 3 minutes, the mapping agent's entry for 4.4.4.4 expires:

```
R3#show ip pim rp mapping
Group(s) 239.2.0.0/16
  RP 2.2.2.2 (?), v2v1
    Info source: 1.1.1.1 (?), elected via Auto-RP

R1#show ip mroute 239.1.1.1
(*, 239.1.1.1), 00:01:02/stopped, RP 0.0.0.0, flags: D
  Incoming interface: Null, RPF nbr 0.0.0.0
  Outgoing interface list:
    GigabitEthernet0/3, Forward/Sparse-Dense, 00:01:02/stopped
    GigabitEthernet0/2, Forward/Sparse-Dense, 00:01:02/stopped
    GigabitEthernet0/1, Forward/Sparse-Dense, 00:01:02/stopped
```

239.1.1.1 now has **no RP**, so on Sparse-Dense interfaces it **falls back to Dense Mode** (flag **D**, RP 0.0.0.0): flood and prune again, with R4 receiving the stream every 3 minutes. RCV still gets its traffic, but the network silently became a Dense Mode network for that group. This is the classic risk of Sparse-Dense Mode: **losing the RP turns sparse groups into dense ones**. `no ip pim dm-fallback` blocks the fallback.

Task 5

On **all** routers:

```
R1(config)#interface range GigabitEthernet0/0 - 3 , Loopback0
R1(config-if-range)#ip pim sparse-mode
R1(config-if-range)#exit
R1(config)#ip pim autorp listener
```

And on R4:

```
R4(config)#ip pim send-rp-announce Loopback0 scope 16
```

```
R3#show ip pim rp mapping | include RP|Group
Group(s) 224.0.0.0/4
  RP 4.4.4.4 (?), v2v1
Group(s) 239.2.0.0/16
  RP 2.2.2.2 (?), v2v1
```

```
R1#show ip mroute 239.1.1.1 | include RP
(*, 239.1.1.1), 00:00:31/stopped, RP 4.4.4.4, flags: S
```

`ip pim autorp listener` lets Sparse Mode interfaces flood **only 224.0.1.39 and 224.0.1.40** in Dense Mode. Auto-RP keeps working, but no other group can ever fall back to Dense Mode. Remove the RP and a group just has no RP (no forwarding), instead of quietly flooding. This is the recommended Auto-RP design today.

Task 6

Remove Auto-RP from R1, R2 and R4:

```
R1(config)#no ip pim send-rp-discovery Loopback0 scope 16
R2(config)#no ip pim send-rp-announce Loopback0 scope 16 group-list AUTORP-R2
R4(config)#no ip pim send-rp-announce Loopback0 scope 16
```

Configure BSR:

```
R1(config)#ip pim bsr-candidate Loopback0
R2(config)#ip pim rp-candidate Loopback0 priority 200
R4(config)#ip pim rp-candidate Loopback0 priority 10
```

Command	Meaning
`bsr-candidate Loopback0`	Candidate BSR. The **highest BSR priority** wins (default 0), then the highest IP
`rp-candidate Loopback0 priority 10`	Candidate RP: unicasts a C-RP advertisement to the elected BSR. For C-RPs the **lowest priority value wins**

```
R3#show ip pim bsr-router
PIMv2 Bootstrap information
  BSR address: 1.1.1.1 (?)
  Uptime:      00:02:13, BSR Priority: 0, Hash mask length: 0
  Expires:     00:01:57

R3#show ip pim rp mapping
Group(s) 224.0.0.0/4
  RP 4.4.4.4 (?), v2
    Info source: 1.1.1.1 (?), via bootstrap, priority 10, holdtime 150
         Uptime: 00:01:40, expires: 00:02:09
  RP 2.2.2.2 (?), v2
    Info source: 1.1.1.1 (?), via bootstrap, priority 200, holdtime 150
         Uptime: 00:01:41, expires: 00:02:08

R3#show ip pim rp-hash 239.1.1.1
  RP 4.4.4.4 (?), v2
    Info source: 1.1.1.1 (?), via bootstrap, priority 10, holdtime 150
```

The difference from Auto-RP: the BSR **does not choose the RP**. It floods the whole **RP set** (both candidates) hop by hop in Bootstrap messages to 224.0.0.13 every 60 seconds, and **every router computes the RP itself** with the same algorithm, so they all agree:

1. Lowest **C-RP priority** wins: here 4.4.4.4 (10 beats 200). This decides it in this lab.
2. On a priority tie: the **hash** of the group address, using the hash mask length, spreads groups across the RPs.
3. On a hash tie: the **highest IP** wins.

`show ip pim rp-hash <group>` shows the result of that calculation for one group. BSR is the IETF standard (RFC 5059) and is part of PIMv2, so it needs no Dense Mode groups at all: the Bootstrap messages are link-local PIM messages.

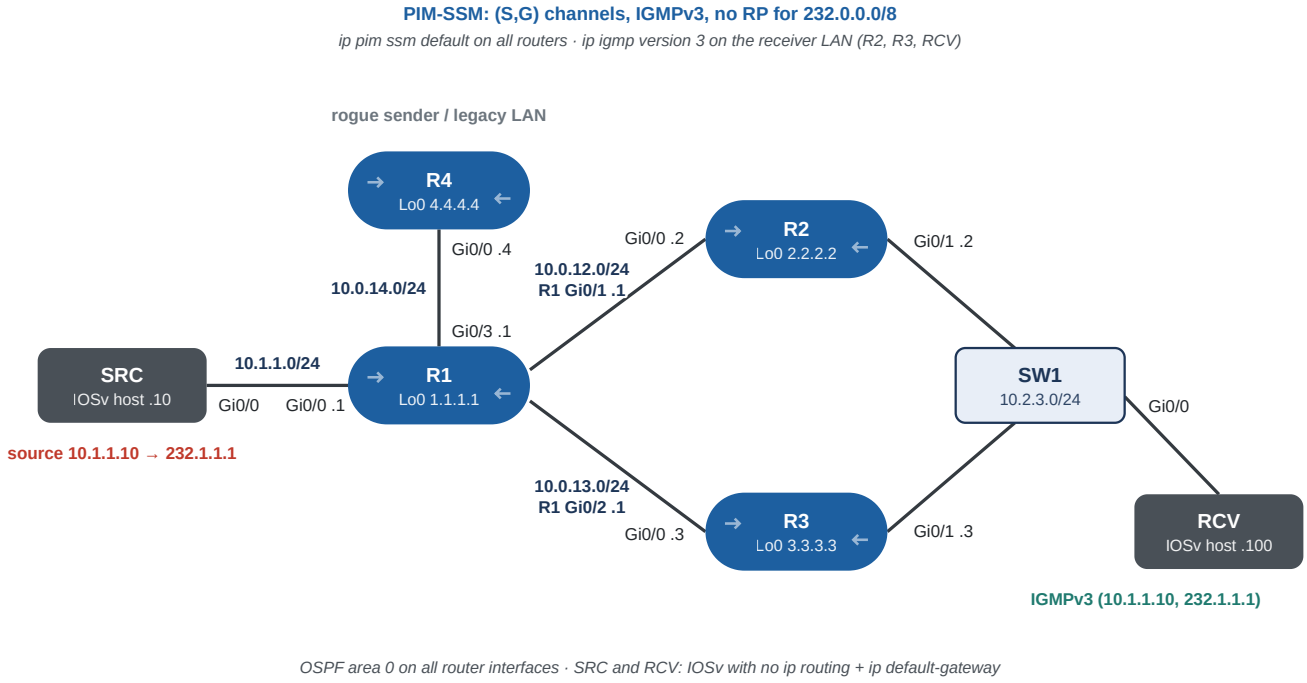
PIM Lab 04: Multicast - PIM Source-Specific Multicast

Lab Objective:

The focus of this lab is to understand PIM Source-Specific Multicast (SSM) in Cisco IOS routers. Additional technologies include IGMPv3 source-specific joins, the SSM group range, SSM and ASM groups running side by side, protection against unwanted sources, and SSM mapping for IGMPv2 receivers.

Lab Topology:

The lab network topology is illustrated below:



IMPORTANT NOTE

Continue from the end of PIM Lab 03: all interfaces run Sparse Mode, BSR elects RP 4.4.4.4, and R2 is the DR on the receiver LAN (DR priority 100). Leave BSR running. SSM needs no RP, and this lab shows that the RP is simply ignored for the SSM range.

Task 1

Enable PIM-SSM for the default SSM range on all routers. Configure IGMPv3 on both router interfaces on the receiver LAN and on RCV.

Task 2

Configure RCV to receive group 232.1.1.1 **only from SRC**. SRC sends to 232.1.1.1. Verify that RCV answers. Verify the IGMP state on the receiver LAN and the multicast routing state on R2, R1 and R4. Explain the flags, and explain why R4 has no state for this group.

Task 3

R4 now behaves as an unwanted source: it sends to 232.1.1.1 from its Loopback0. Verify that RCV does not receive it, and explain which mechanism blocks it. Then verify that group 239.1.1.1 still uses the RP.

Task 4

The applications team will also use groups 239.232.0.0/16 for SSM. Configure all routers so that both 232.0.0.0/8 and 239.232.0.0/16 are SSM groups. RCV joins 239.232.1.1 from SRC only. Verify that this group no longer uses the RP.

Task 5

R4 gets a legacy receiver LAN, simulated by Loopback1 with 10.4.4.4/24, that only speaks IGMPv2. Its hosts need group 232.1.1.2 from SRC, and the hosts can't be changed. Configure R4 so that the receivers on Loopback1 get this stream, without DNS lookups. Simulate the receiver with a static IGMP join on Loopback1, and verify the result on R4 and R1.

Lab Validation:

The outputs below are trimmed. Timers, counters and uptimes will differ in your lab.

Task 1

```
R1(config)#ip pim ssm default
R2(config)#ip pim ssm default
R2(config)#interface GigabitEthernet0/1
R2(config-if)#ip igmp version 3
R3(config)#ip pim ssm default
R3(config)#interface GigabitEthernet0/1
R3(config-if)#ip igmp version 3
R4(config)#ip pim ssm default
RCV(config)#interface GigabitEthernet0/0
RCV(config-if)#ip igmp version 3
```

`ip pim ssm default` makes **232.0.0.0/8** (the IANA SSM range) an SSM range on that router. Configure it on **every** router, otherwise a router outside the range treats 232.x.x.x as an ordinary ASM group and looks for an RP.

IGMPv3 is only needed where hosts are: SSM needs the receiver to say **which source** it wants, and IGMPv1/v2 can't carry a source. Between routers it's plain PIM (S,G) joins.

```
R2#show ip igmp interface GigabitEthernet0/1 | include version
  IGMP is enabled on interface
  Current IGMP host version is 3
  Current IGMP router version is 3
```

Task 2

```
RCV(config)#interface GigabitEthernet0/0
RCV(config-if)#ip igmp join-group 232.1.1.1 source 10.1.1.10
SRC#ping 232.1.1.1 repeat 5
Reply to request 0 from 10.2.3.100, 6 ms
Reply to request 1 from 10.2.3.100, 2 ms
```

```
R2#show ip igmp groups 232.1.1.1 detail
Interface:      GigabitEthernet0/1
Group:          232.1.1.1
Flags:          SSM
Group mode:     INCLUDE
Last reporter:  10.2.3.100
Group source list:
  Source Address   Uptime    v3 Exp   CSR Exp   Fwd  Flags
  10.1.1.10        00:01:12  00:02:47  stopped   Yes  R
```

RCV sent an IGMPv3 report in **INCLUDE** mode: "I want 232.1.1.1, only from 10.1.1.10". That pair (S,G) is called a **channel**.

```
R2#show ip mroute 232.1.1.1
(10.1.1.10, 232.1.1.1), 00:01:12/00:02:47, flags: sTI
  Incoming interface: GigabitEthernet0/0, RPF nbr 10.0.12.1
  Outgoing interface list:
    GigabitEthernet0/1, Forward/Sparse, 00:01:12/00:02:47

R1#show ip mroute 232.1.1.1
(10.1.1.10, 232.1.1.1), 00:01:11/00:03:18, flags: sT
  Incoming interface: GigabitEthernet0/0, RPF nbr 0.0.0.0
  Outgoing interface list:
    GigabitEthernet0/1, Forward/Sparse, 00:01:11/00:03:18

R4#show ip mroute 232.1.1.1
Group 232.1.1.1 not found
```

Flag	Meaning
s	SSM group
I	Received a source-specific host report (IGMPv3)
T	Traffic is arriving on the shortest-path tree

- There is **no (*,G) entry** and **no RP** in the output. The DR (R2) knew the source from the report, so it sent an **(S,G) join straight toward 10.1.1.10** (its RPF interface toward the source).
- R1 has no **F** flag: in SSM the first-hop router **never registers**, so no Register or Register-Stop.
- R4 has no state at all. The RP isn't part of an SSM tree, so it doesn't matter that BSR still advertises 4.4.4.4 for 224.0.0.0/4.

Task 3

```
R4#ping 232.1.1.1 source Loopback0 repeat 5
.....
R2#show ip mroute 232.1.1.1 | include \(
(10.1.1.10, 232.1.1.1), 00:04:40/00:02:59, flags: sTI
```

No reply. The only state on the receiver side is **(10.1.1.10, 232.1.1.1)**. A packet from 4.4.4.4 to 232.1.1.1 matches no forwarding entry, and since 232/8 is in the SSM range, no router will register it to an RP or build a shared tree for it. Nobody ever sent a join for (4.4.4.4, 232.1.1.1), so the traffic dies on R4.

That's the security benefit of SSM: a host receives **only the sources it asked for**. In ASM, any source sending to the group reaches the RP and then every receiver.

ASM still works for the other groups. With SRC sending to 239.1.1.1, the receiver DR uses the RP from BSR:

```
R2#show ip mroute 239.1.1.1 | include RP
(*, 239.1.1.1), 03:10:41/stopped, RP 4.4.4.4, flags: SJC
```

Task 4

On **all** routers:

```
R1(config)#ip access-list standard SSM-RANGE
R1(config-std-nacl)#permit 232.0.0.0 0.255.255.255
R1(config-std-nacl)#permit 239.232.0.0 0.0.255.255
R1(config-std-nacl)#exit
R1(config)#ip pim ssm range SSM-RANGE
```

`ip pim ssm range <acl>` **replaces** the default range. That is why 232.0.0.0/8 must be in the ACL too, or it stops being SSM.

```
RCV(config)#interface GigabitEthernet0/0
RCV(config-if)#ip igmp join-group 239.232.1.1 source 10.1.1.10
SRC#ping 239.232.1.1 repeat 5
Reply to request 0 from 10.2.3.100, 5 ms

R2#show ip mroute 239.232.1.1
(10.1.1.10, 239.232.1.1), 00:00:22/00:02:37, flags: sTI
  Incoming interface: GigabitEthernet0/0, RPF nbr 10.0.12.1
  Outgoing interface list:
    GigabitEthernet0/1, Forward/Sparse, 00:00:22/00:02:37
```

Flag **s**, no (*,G), no RP. BSR still maps 224.0.0.0/4 to 4.4.4.4, but the SSM range always wins: for a group in the SSM range, the router doesn't look up an RP at all.

Task 5

```
R4(config)#interface Loopback1
R4(config-if)#ip address 10.4.4.4 255.255.255.0
R4(config-if)#ip pim sparse-mode
R4(config-if)#exit
R4(config)#ip access-list standard SSM-MAP-GROUPS
R4(config-std-nacl)#permit 232.1.1.2
R4(config-std-nacl)#exit
R4(config)#ip igmp ssm-map enable
R4(config)#no ip igmp ssm-map query dns
R4(config)#ip igmp ssm-map static SSM-MAP-GROUPS 10.1.1.10
R4(config)#interface Loopback1
R4(config-if)#ip igmp static-group 232.1.1.2 source ssm-map
```

Command	Meaning
`ip igmp ssm-map enable`	Turn on SSM mapping on the last-hop router
`no ip igmp ssm-map query dns`	Use only the static mappings. DNS lookups are on by default
`ip igmp ssm-map static <acl> <source>`	Groups matching the ACL map to that source
`ip igmp static-group <group> source ssm-map`	Static join that uses the mapping (stands in for a real IGMPv2 host)

```
R4#show ip igmp ssm-mapping 232.1.1.2
Group address: 232.1.1.2
Database     : Static
```

```
Source list  : 10.1.1.10

R4#show ip mroute 232.1.1.2
(10.1.1.10, 232.1.1.2), 00:00:41/stopped, flags: sCT
  Incoming interface: GigabitEthernet0/0, RPF nbr 10.0.14.1
  Outgoing interface list:
    Loopback1, Forward/Sparse, 00:00:41/00:02:18

R1#show ip mroute 232.1.1.2
(10.1.1.10, 232.1.1.2), 00:00:40/00:02:49, flags: sT
  Incoming interface: GigabitEthernet0/0, RPF nbr 0.0.0.0
  Outgoing interface list:
    GigabitEthernet0/3, Forward/Sparse, 00:00:40/00:02:49
```

Without mapping, an IGMPv2 report for 232.1.1.2 says "I want the group" with no source. In the SSM range there's no RP and no shared tree, so the router **can't use it** and ignores it. **SSM mapping** fixes this on the **last-hop router**: R4 looks up the group in its mapping database, gets 10.1.1.10, and sends a normal **(S,G) join** upstream. The rest of the network sees plain SSM. The hosts don't change. Start a ping to 232.1.1.2 on SRC and check `show ip mroute 232.1.1.2 count` on R4: the packet counter increases.

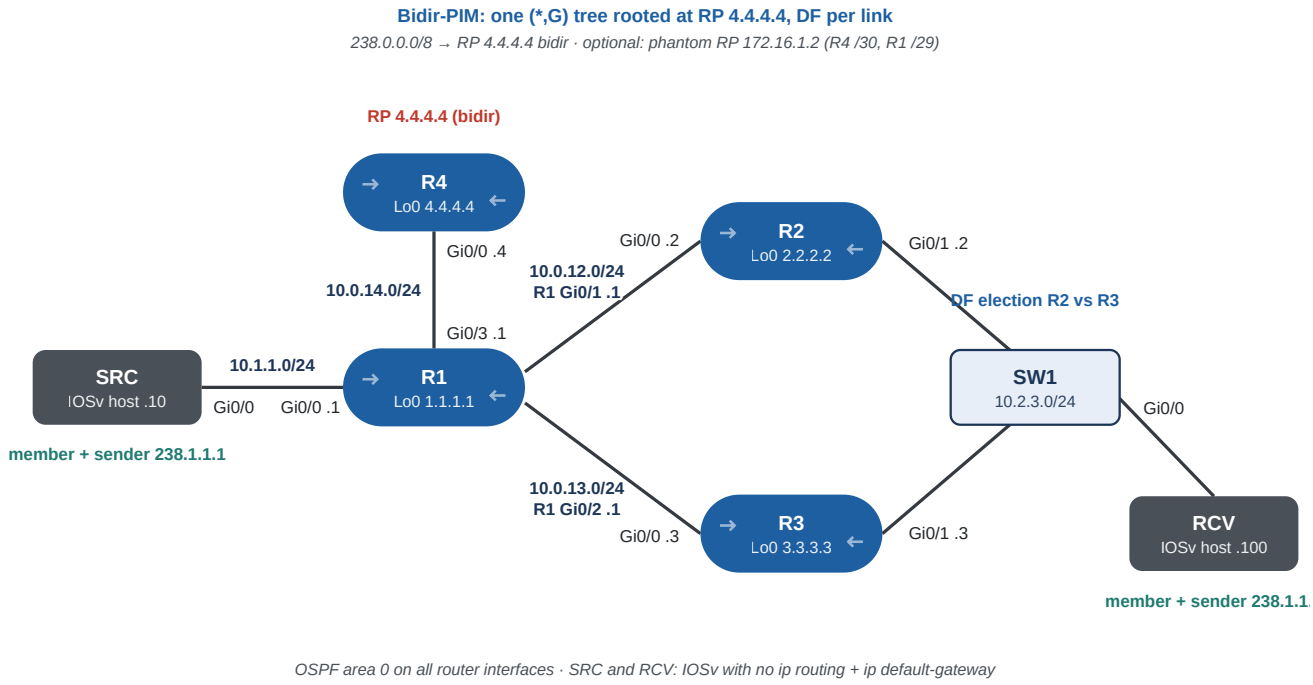
PIM Lab 05: Multicast - Bidirectional PIM

Lab Objective:

The focus of this lab is to understand Bidirectional PIM (Bidir-PIM) in Cisco IOS routers. Additional technologies include the bidir RP, Designated Forwarder (DF) election, the bidir (*,G) tree, many-to-many applications, and RP redundancy with a Phantom RP.

Lab Topology:

The lab network topology is illustrated below:



IMPORTANT NOTE

Continue from the end of PIM Lab 04. Stop all pings first. Remove BSR from R1, R2 and R4 (`no ip pim bsr-candidate Loopback0` , `no ip pim rp-candidate Loopback0`). The SSM configuration can stay, because 238.0.0.0/8 isn't in the SSM range.

Task 1

Enable Bidir-PIM on all routers. Configure R4's Loopback0 as a static **bidirectional** RP for groups 238.0.0.0/8 only, on every router. Verify the group-to-RP mapping.

Task 2

Verify the Designated Forwarder on every link in the network. For each link, explain why that router won the election. Write the result on your topology drawing.

Task 3

SRC and RCV are both members of group 238.1.1.1 and both send to it, like two sites of a video conference. Configure it, and verify that each host receives the other's traffic. Verify the multicast routing state on R1 and on the receiver LAN routers. Which router forwards the group onto the receiver LAN?

Task 4

Verify that the first-hop router never registers the source and that no (S,G) state is built. Explain where traffic from SRC goes first, and why R4 receives the stream even though it has no receivers.

Task 5

Make R2 the Designated Forwarder on the receiver LAN without changing any PIM setting. Verify the result, then undo the change.

Task 6

Make the RP redundant with a **Phantom RP**:

- The RP address is 172.16.1.2, which is not configured on any router
- R4 is the primary RP with Loopback2 172.16.1.1/30
- R1 is the backup RP with Loopback2 172.16.1.1/29

Verify which router is the active RP for 238.1.1.1. Shut down R4's Loopback2 and verify that the conference keeps working through R1.

Lab Validation:

The outputs below are trimmed. Timers, counters and uptimes will differ in your lab.

Task 1

On **all** routers:

```
R1(config)#ip pim bidir-enable
R1(config)#ip access-list standard BIDIR-GROUPS
R1(config-std-nacl)#permit 238.0.0.0 0.255.255.255
R1(config-std-nacl)#exit
R1(config)#ip pim rp-address 4.4.4.4 BIDIR-GROUPS bidir
```

`ip pim bidir-enable` allows bidir on the router; on some IOS releases it's already on and the line doesn't appear in the config. The keyword **bidir** at the end of `rp-address` makes this RP bidirectional for the groups in the ACL. Every router must agree that the group is bidir, otherwise one router builds an ASM tree and another a bidir tree for the same group.

```
R3#show ip pim rp mapping
PIM Group-to-RP Mappings

Acl: BIDIR-GROUPS, Static, Bidir Mode
    RP: 4.4.4.4 (?)
```

Task 2

```
R1#show ip pim interface df
* implies this system is the DF
Interface                RP               DF Winner        Metric     Uptime
GigabitEthernet0/0       4.4.4.4          *10.1.1.1        2          00:02:11
GigabitEthernet0/1       4.4.4.4          *10.0.12.1       2          00:02:11
GigabitEthernet0/2       4.4.4.4          *10.0.13.1       2          00:02:11
GigabitEthernet0/3       4.4.4.4           10.0.14.4       0          00:02:11

R2#show ip pim interface df
Interface                RP               DF Winner        Metric     Uptime
GigabitEthernet0/0       4.4.4.4           10.0.12.1       2          00:02:09
GigabitEthernet0/1       4.4.4.4           10.2.3.3        3          00:02:09
```

Link	DF	Why
R1–R4	R4	Owns the RP address: metric 0. This link is the **RP Link (RPL)**, R4's Lo0 too
SRC LAN	R1	Only PIM router on the link
R1–R2, R1–R3	R1	Metric 2 to the RP beats metric 3
Receiver LAN	R3	R2 and R3 tie (both metric 3), so the **highest IP** wins: 10.2.3.3

The DF election compares the **unicast route to the RP**: the lowest AD wins, then the lowest metric, then the highest IP. It's the same order as a PIM assert, but it runs **once per link per RP** when PIM starts, not when traffic arrives. Bidir needs it because the tree carries traffic in both directions. Without one forwarder per link, packets going up toward the RP could be copied back down and loop.

Task 3

```
SRC(config)#interface GigabitEthernet0/0
SRC(config-if)#ip igmp join-group 238.1.1.1
RCV(config)#interface GigabitEthernet0/0
RCV(config-if)#ip igmp join-group 238.1.1.1
SRC#ping 238.1.1.1 repeat 5
Reply to request 0 from 10.2.3.100, 7 ms
RCV#ping 238.1.1.1 repeat 5
Reply to request 0 from 10.1.1.10, 6 ms
```

```
R1#show ip mroute 238.1.1.1
(*, 238.1.1.1), 00:01:40/00:02:49, RP 4.4.4.4, flags: BC
  Bidir-Upstream: GigabitEthernet0/3, RPF nbr 10.0.14.4
  Outgoing interface list:
    GigabitEthernet0/2, Forward/Sparse, 00:01:40/00:02:49
    GigabitEthernet0/0, Forward/Sparse, 00:01:40/00:02:21
    GigabitEthernet0/3, Bidir-Upstream/Sparse, 00:01:40/stopped

R3#show ip mroute 238.1.1.1
(*, 238.1.1.1), 00:01:38/00:02:41, RP 4.4.4.4, flags: BC
  Bidir-Upstream: GigabitEthernet0/0, RPF nbr 10.0.13.1
  Outgoing interface list:
    GigabitEthernet0/1, Forward/Sparse, 00:01:38/00:02:41
    GigabitEthernet0/0, Bidir-Upstream/Sparse, 00:01:38/stopped
```

- Flag **B** = bidir group. There is **no incoming interface**, only a **Bidir-Upstream** interface toward the RP, which is also in the outgoing list. The same (*,G) entry is used to **send** up and **receive** down.
- **R3 forwards the group onto the receiver LAN**, even though R2 is the DR (priority 100 from Lab 02). In bidir, the **DF** handles local receivers and forwards traffic onto and off the link. The DR plays no role for

bidir groups.

- R2 still sees the IGMP reports (it's the querier), but it isn't the DF on the LAN, so it neither forwards the group onto the LAN nor picks up the hosts' traffic.

Task 4

```
R1#debug ip pim
SRC#ping 238.1.1.1 repeat 5
R1#show ip mroute 238.1.1.1 | include \(
(*, 238.1.1.1), 00:04:12/00:02:31, RP 4.4.4.4, flags: BC
R1#undebug all
```

No Register messages appear in the debug, and there is **no (S,G) entry**. Bidir uses only the shared tree:

1. R1 is the DF on the SRC LAN, so it accepts the packet and forwards it **up toward the RP** (Bidir-Upstream Gi0/3) **and** down every other branch with receivers (Gi0/2 toward R3).
2. R4 receives the stream because traffic **always travels all the way to the RP link**, even when it isn't needed there. That's the cost of bidir: no shortcuts and no SPT.
3. The benefit: each router keeps **one (*,G) entry per group** no matter how many sources there are. A conference with 500 senders in ASM would mean 500 (S,G) entries on every router on the path. That's why bidir is used for many-to-many applications.

Task 5

```
R3(config)#interface GigabitEthernet0/0
R3(config-if)#ip ospf cost 10
R2#show ip pim interface GigabitEthernet0/1 df
Interface                RP               DF Winner        Metric     Uptime
GigabitEthernet0/1       4.4.4.4          *10.2.3.2        3          00:00:12
```

R3's route to 4.4.4.4 now costs 12, so R2 wins with 3 and takes over the LAN. Verify with `show ip mroute 238.1.1.1` on R2: Gi0/1 is now in its outgoing list. The receiver LAN is passive in OSPF, so R3 can't use it as a cheaper transit path through R2. Undo the change:

```
R3(config-if)#no ip ospf cost 10
```

Task 6

On R4 and R1:

```
R4(config)#interface Loopback2
R4(config-if)#ip address 172.16.1.1 255.255.255.252
R4(config-if)#ip ospf network point-to-point
R4(config-if)#ip pim sparse-mode
R1(config)#interface Loopback2
R1(config-if)#ip address 172.16.1.1 255.255.255.248
R1(config-if)#ip ospf network point-to-point
R1(config-if)#ip pim sparse-mode
```

On **all** routers:

```
R1(config)#no ip pim rp-address 4.4.4.4 BIDIR-GROUPS bidir
R1(config)#ip pim rp-address 172.16.1.2 BIDIR-GROUPS bidir
```

`ip ospf network point-to-point` makes OSPF advertise the loopback with its **real mask**. By default OSPF advertises a loopback as a /32, and then both routers would advertise the same 172.16.1.1/32. The overlapping IP 172.16.1.1 on two loopbacks is fine, because the OSPF router IDs were set manually in Lab 01.

```
R3#show ip route 172.16.1.2
Routing entry for 172.16.1.0/30
  Known via "ospf 1", distance 110, metric 3, type intra area
  * 10.0.13.1, from 4.4.4.4, via GigabitEthernet0/0

R1#show ip mroute 238.1.1.1 | include RP|Upstream
(*, 238.1.1.1), 00:00:51/00:02:38, RP 172.16.1.2, flags: BC
  Bidir-Upstream: GigabitEthernet0/3, RPF nbr 10.0.14.4
```

Nobody owns 172.16.1.2, so it never answers a ping. That's expected. The routers only need a **route** toward it, and the **longest match wins**: the /30 from R4 beats the /29 from R1, so R4's Loopback2 is the RP link and R4 is the active RP.

```
R4(config)#interface Loopback2
R4(config-if)#shutdown
R1#show ip route 172.16.1.2 | include entry|via
Routing entry for 172.16.1.0/29
  directly connected, via Loopback2
R1#show ip mroute 238.1.1.1 | include RP|Upstream
(*, 238.1.1.1), 00:03:12/00:02:51, RP 172.16.1.2, flags: BC
  Bidir-Upstream: Loopback2, RPF nbr 0.0.0.0
SRC#ping 238.1.1.1 repeat 5
Reply to request 0 from 10.2.3.100, 5 ms
```

The /30 disappeared from OSPF, the /29 is now the longest match, and R1's Loopback2 became the RP link. Failover is only as fast as **OSPF convergence**. There's no RP state to move, because in bidir the RP keeps no source state and receives no Registers. It's just the root of the tree. That's also why Phantom RP **only works with bidir**: in ASM, Register messages must reach a real router that owns the RP address. For ASM, the redundancy tool is Anycast RP with MSDP.